

Rudra: Shield Against Deception

Plan of Action — PSBs Hackathon Series 2026

Team Bhadra | Government of India, Ministry of Finance, Department of Financial Services

Document Version 1.0 | Prepared for Hackathon Evaluation

Member	Role	Area of Responsibility
Satyadev Suvesh	Agentic AI	LLM Investigator Copilot, RAG pipeline, Auto FIU Evidence generation
Anant Asati	Agentic AI	LLM integration, KYC delta reasoning, Qwen local explainer
Prashant Gautam	Graphs & ML	PyTorch Geometric, FraudGT, BDH Model, XGBoost, 5-pattern parallel detector
Yash Kumar Maru	Web Development	React/Vite dashboard, FastAPI backend, Database management

Contents

1	Executive Summary	3
2	System Architecture	3
2.1	Streaming and Ingestion Layer	3
2.2	Heterogeneous Graph Engine	4
2.3	Graph ML and Detection Layer	4
2.4	Agentic AI Layer	4
2.5	Backend, Data, and Frontend	4
3	Phase-wise Implementation Plan	5
3.1	Phase 1: Environment Setup and Data Ingestion	5
3.2	Phase 2: Core Graph ML and Detection Engine	5
3.3	Phase 3: Agentic AI, RAG Pipeline, and FIU Evidence Generation	6
3.4	Phase 4: Frontend Dashboard and Final Deployment	7
4	Risk Mitigation Strategy	9
5	Testing and Validation	9
5.1	ML Benchmark Validation	9
5.2	Latency Validation	10
5.3	FIU Evidence Package Validation	10
5.4	Compliance Validation	11

6 References

11

1. Executive Summary

Rudra is a real-time anti-money laundering (AML) and fraud detection platform built for India's public sector banks. The core problem it solves is the industry-wide reliance on T+1 batch analysis, where fraud is only discovered the morning after it occurs. By the time a batch job flags a suspicious round-trip, the funds have already been layered across multiple accounts.

Rudra replaces this paradigm with a **streaming-native fund flow graph** built on Pathway, updated per transaction with an end-to-end detection latency target of **under 200 milliseconds**. Five parallel Graph ML detectors run continuously on the live graph, covering layering, round-tripping, structuring, dormant account reactivation, and KYC profile mismatch. A tool-calling LLM investigator copilot compresses case investigation from days to seconds, and autonomously generates a fully compliant FIU evidence package (SAR narrative, subgraph export, transaction chain PDF, and PMLA section citations) in under 5 seconds.

DPI-Native Approach

Rudra is explicitly architected for India's Digital Public Infrastructure. It integrates the **Account Aggregator (AA) framework** for consent-based real-time financial data ingestion, aligns with the RBI's Real-time Fraud Risk Management mandate, and enforces data minimisation principles under the **DPDP Act 2023**. Regulatory citations are auto-generated into every FIU evidence package, eliminating manual compliance drafting.

The ML backbone is benchmarked on IBM AML-HI-Large (2.1M nodes, 180M transactions), where the FraudGT + BDH ensemble achieves an F1 score of **0.72**, compared to 0.42 for a baseline XGBoost model.

2. System Architecture

The Rudra architecture is organised into five layers: Ingestion, Graph Engine, ML Detection, Agentic AI, and Backend/Frontend. Data flows unidirectionally from raw transaction streams through to investigator-ready evidence packages.

2.1. Streaming and Ingestion Layer

- **Pathway**: Core streaming engine. Provides stateful, incremental graph updates per transaction. Replaces overnight batch snapshots with sub-200ms incremental updates.
- **Kafka / NATS**: Message broker for high-throughput transaction event ingestion from core banking, UPI, and Account Aggregator streams.
- **Apache Flink**: Supplementary stream processing for windowed feature aggregation (1h, 6h, 24h, 7-day rolling windows).
- **Account Aggregator (AA) Framework**: Consent-based real-time financial data pull. Provides normalised transaction feeds across banks under a single DPDP-compliant channel.
- **DiliSense API**: External KYC data enrichment for profile delta computation.

2.2. Heterogeneous Graph Engine

The live transaction graph is modelled as a **PyTorch Geometric (PyG) HeteroData** object.

- **Nodes:** Accounts, merchants, branches.
- **Edges:** Carry amount, timestamp, channel, and branch metadata.
- **Update mechanism:** Pathway connectors push incremental node and edge additions on each incoming transaction. No full graph recomputation required.

2.3. Graph ML and Detection Layer

Five specialised detectors run in parallel on the live graph:

1. **Layering:** FraudGT (Graph Transformer) path scoring over multi-hop fund chains.
2. **Round-tripping:** Johnson's cycle enumeration algorithm over directed transaction cycles.
3. **Structuring:** Sliding-window threshold analysis for sub-reporting-limit splits.
4. **Dormant Account Reactivation:** Z-score spike detection on account activity timelines.
5. **KYC Profile Mismatch:** LLM reasoning over KYC delta between declared and observed profiles.

The **FraudGT + BDH model ensemble** underpins detectors 1 and 2. FraudGT identifies structural anomalies (cyclic payments, chain depth, motif patterns). The BDH (Dragon Hatchling) model generates human-readable fraud reason strings that are forwarded directly to the LLM for SAR narrative drafting. XGBoost is retained as a baseline comparator and fallback for feature-based scoring.

2.4. Agentic AI Layer

- **Gemini 2.0 Flash:** Primary LLM powering the investigator copilot. Executes tool calls over the live graph to answer investigator queries in plain language.
- **Tool suite:** `trace_funds()`, `find_cycles()`, `explain_alert()`, `get_profile_delta()`.
- **ChromaDB:** Vector database backing the RAG pipeline. Stores regulatory documents (PMLA, RBI circulars, DPDP Act) for grounded SAR narrative generation.
- **all-MiniLM-L6-v2:** Sentence embedding model used to index and retrieve relevant regulatory passages during SAR drafting.
- **Qwen (local):** Lightweight local explainer for offline or low-latency KYC delta reasoning tasks that do not require full Gemini inference.

2.5. Backend, Data, and Frontend

- **FastAPI (Python):** REST API serving the dashboard, exposing alert endpoints, copilot query interface, and FIU package download.
- **PostgreSQL / AWS RDS:** Primary transactional store for account profiles, alert records, and case management state.
- **SQLite audit DB:** Immutable local audit trail for all LLM reasoning steps (full reasoning chain preserved for regulatory review).

- **AWS:** Cloud deployment target. Standard EC2/RDS stack; no proprietary AWS ML services required.
- **React + Vite:** Investigator dashboard. Displays live alert feed, fund flow graph visualisation, copilot chat interface, and FIU package export.

3. Phase-wise Implementation Plan

The hackathon implementation is divided into four logical phases. Each phase has a defined set of deliverables and owner assignments based on team member expertise.

3.1. Phase 1: Environment Setup and Data Ingestion

Goal: Establish a working local development environment, ingest transaction data from the IBM AML-HI-Large dataset, and publish it through the Pathway streaming pipeline.

Tasks

#	Owner	Task
1.1	Yash	Provision local dev environment: Docker Compose for Kafka, PostgreSQL, ChromaDB. Configure AWS credentials and RDS instance.
1.2	Yash	Set up FastAPI project skeleton with health-check endpoint. Configure SQLite audit DB schema.
1.3	Satyadev	Implement Pathway connector to read IBM AML-HI-Large CSV and emit transaction events to Kafka topic.
1.4	Satyadev	Validate end-to-end event flow: Kafka consumer acknowledgment, Pathway stateful window confirmation.
1.5	Anant	Integrate DiliSense API mock for KYC profile enrichment per account node. Store enriched profiles in PostgreSQL.
1.6	Prashant	Define PyG HeteroData schema: node types (account, merchant, branch), edge types (transfer, payment), and edge attribute fields.

Deliverables

- Live Kafka topic receiving parsed transaction events from IBM AML dataset.
- Pathway streaming job running with stateful window operators active.
- PyG HeteroData schema validated against a 10,000-transaction sample.
- FastAPI server running with database connectivity confirmed.

3.2. Phase 2: Core Graph ML and Detection Engine

Goal: Build and validate the live heterogeneous graph and deploy all five pattern detectors on top of it using the FraudGT + BDH ensemble.

Tasks

#	Owner	Task
2.1	Prashant	Implement incremental graph builder: Pathway callback appends new nodes and edges to PyG HeteroData on each transaction event.
2.2	Prashant	Build windowed feature aggregation: 1h, 6h, 24h, 7-day rolling aggregates per account node (txn count, total amount, unique counterparties).
2.3	Prashant	Implement Layering detector using FraudGT path scoring. Threshold calibrated against IBM AML-HI-Large labels.
2.4	Prashant	Implement Round-tripping detector using Johnson's directed cycle enumeration.
2.5	Prashant	Implement Structuring detector using sliding-window threshold analysis (sub-50k splits within 24h window).
2.6	Prashant	Implement Dormant Account Reactivation detector using Z-score spike on per-account activity timelines.
2.7	Anant	Implement KYC Profile Mismatch detector: extract KYC delta between DiliSense profile and observed transaction behaviour; pass delta string to Qwen for binary mismatch classification.
2.8	Prashant	Integrate BDH model: attach human-readable fraud reason string to each positive alert from detectors 1 and 2.
2.9	Prashant	Run five detectors in parallel using Python <code>concurrent.futures</code> or <code>asyncio</code> task group; merge alert results into a unified alert event.
2.10	Satyadev	Expose <code>POST /alerts</code> endpoint in FastAPI to receive unified alert events from detection layer and persist to PostgreSQL.

Deliverables

- Live graph updates confirmed at <200ms per transaction on a 10k-txn replay.
- All five detectors producing alerts on known-positive samples from IBM AML-HI-Large.
- FraudGT F1 score ≥ 0.70 on held-out IBM AML test split.
- Unified alert schema persisted to PostgreSQL with BDH reason string attached.

3.3. Phase 3: Agentic AI, RAG Pipeline, and FIU Evidence Generation

Goal: Deploy the Gemini-powered investigator copilot, populate the ChromaDB RAG store with regulatory documents, and implement the autonomous FIU evidence package generator.

Tasks

#	Owner	Task
3.1	Satyadev	Ingest regulatory corpus into ChromaDB: PMLA sections, RBI FRM circular, DPDP Act 2023, AA framework documentation. Embed using all-MiniLM-L6-v2.
3.2	Satyadev	Implement tool functions callable by Gemini: <code>trace_funds()</code> , <code>find_cycles()</code> , <code>explain_alert()</code> , <code>get_profile_delta()</code> . Each tool queries the live graph or PostgreSQL.
3.3	Satyadev	Build Gemini 2.0 Flash tool-calling agent: register tool schemas, implement streaming response handler, log each reasoning step to SQLite audit DB.
3.4	Anant	Implement <code>explain_alert()</code> using Qwen local explainer as a lightweight fallback when Gemini API latency exceeds threshold.
3.5	Anant	Implement <code>get_profile_delta()</code> tool: computes structured delta between DiliSense KYC fields and observed graph behaviour; formats output for LLM consumption.
3.6	Satyadev	Build FIU evidence package generator: on alert confirmation, trigger SAR narrative generation (Gemini + RAG), subgraph export (NetworkX JSON), transaction chain PDF (ReportLab), and PMLA section citations (ChromaDB retrieval). Target: full package in <5 seconds.
3.7	Satyadev	Expose <code>POST /copilot/query</code> and <code>GET /alerts/{id}/fiu-package</code> endpoints in FastAPI.
3.8	Anant	Implement 70+ rule-based cold-start checks for new accounts with no transaction history. Rules cover amount thresholds, geography, channel type, and time-of-day patterns.

Deliverables

- ChromaDB store populated and retrieval validated against PMLA query samples.
- Gemini copilot answering fund trace queries end-to-end with tool calls logged.
- FIU evidence package generated in <5 seconds on a confirmed alert.
- SAR narrative contains correct PMLA section citations sourced from ChromaDB.
- Cold-start rule engine returning risk scores for zero-history accounts.

3.4. Phase 4: Frontend Dashboard and Final Deployment

Goal: Deliver the investigator-facing React dashboard, complete end-to-end integration testing, and deploy to AWS.

Tasks

#	Owner	Task
4.1	Yash	Build React + Vite dashboard: live alert feed with severity badges, account detail panel, alert timeline.
4.2	Yash	Implement interactive fund flow graph visualisation (D3.js or react-force-graph) rendered from subgraph export JSON.
4.3	Yash	Build LLM copilot chat interface: streaming response display, tool call status indicators, audit trail toggle.
4.4	Yash	Implement FIU package download button: triggers <code>GET /alerts/{id}/fiu-package</code> , streams PDF to browser.
4.5	Satyadev	End-to-end integration test: replay 50k IBM AML transactions through full pipeline. Validate latency, alert accuracy, and FIU package correctness.
4.6	Prashant	Latency profiling: instrument Pathway callback, detector parallel execution, and alert persistence. Identify and fix bottlenecks exceeding 200ms.
4.7	Anant	Model accuracy audit: compute precision, recall, F1 on IBM AML-HI-Large test split. Document results against XGBoost baseline.
4.8	Yash	AWS deployment: EC2 for FastAPI + Pathway, RDS for PostgreSQL, S3 for FIU package storage. Configure environment variables and secrets.
4.9	All	Final demo preparation: prepare 10-minute walkthrough covering live alert detection, copilot query, and FIU package export.

Deliverables

- Fully functional React dashboard connected to live FastAPI backend.
- End-to-end latency $\leq 200\text{ms}$ confirmed on 50k transaction replay.
- AWS deployment accessible via public endpoint.
- Final demo script and presentation materials complete.

4. Risk Mitigation Strategy

Risk	Level	Root Cause	Mitigation
Large-Scale Transaction Graphs	Medium	Full graph recomputation at scale is prohibitive; memory pressure on PyG HeteroData at millions of nodes.	Use Pathway’s incremental update model: only new nodes and edges are appended per transaction. Partition graph by account group shards to distribute memory. Validate at 2M node scale using IBM AML-HI-Large before demo.
Real-Time Data Consent & Privacy	Medium	Account Aggregator pulls require user consent; DPDP Act 2023 mandates data minimisation and purpose limitation.	Use AA framework’s existing consent-token model for all data pulls. Apply field-level minimisation: ingest only transaction amount, timestamp, channel, and counterparty ID. No PII stored in graph edges. All storage subject to retention limits defined in DPDP Act.
New Users with No Transaction History	Low	GNN-based detectors require historical graph structure; cold-start accounts have no edges, making ML scoring unreliable.	70+ rule-based cold-start checks cover amount thresholds, geographic anomaly, channel type, and time-of-day patterns. ML detectors are suppressed for accounts with fewer than 5 historical transactions; rule engine takes over.
Model Accuracy Drift Over Time	Low	Transaction patterns, fraud typologies, and attacker behaviour evolve; a static model degrades in F1 over time.	30-day retraining cycle: investigator feedback (confirmed positives and dismissed false positives) is logged to PostgreSQL and fed back as labelled data for FraudGT fine-tuning. Accuracy metrics are tracked per deployment cohort.

5. Testing and Validation

5.1. ML Benchmark Validation

All classification models are evaluated against the **IBM AML-HI-Large dataset** (2.1M nodes, approximately 180M transactions, IBM Research 2023). The dataset provides

ground-truth labels for known AML patterns including layering, structuring, and round-tripping.

Evaluation Protocol

- **Train/Test Split:** 80/20 stratified split preserving label distribution.
- **Primary Metric:** F1 score (harmonic mean of precision and recall) on the positive (fraudulent) class.
- **Baseline:** XGBoost on hand-crafted features ($F1 = 0.42$ on this dataset).
- **Target:** FraudGT + BDH ensemble $F1 \geq 0.70$.
- **Secondary Metrics:** Precision, Recall, AUC-ROC, and False Positive Rate (FPR). FPR is operationally critical: excessive false positives overload investigators.
- **Per-Detector Validation:** Each of the five parallel detectors is validated independently on its target pattern subset before ensemble integration.

5.2. Latency Validation

The sub-200ms end-to-end latency target covers the full pipeline from transaction event receipt at the Kafka consumer to alert persistence in PostgreSQL.

Instrumentation Plan

1. **Pathway callback timer:** Measure time from Kafka message receipt to graph update commit. Target: $\leq 50\text{ms}$.
2. **Feature aggregation timer:** Measure windowed aggregation compute time. Target: $\leq 30\text{ms}$.
3. **Parallel detector execution timer:** Measure time from feature vector availability to merged alert output. Target: $\leq 80\text{ms}$.
4. **Alert persistence timer:** Measure FastAPI endpoint handler + PostgreSQL write. Target: $\leq 40\text{ms}$.
5. **End-to-end P95 target:** $\leq 200\text{ms}$ across a 50,000-transaction replay.

Load Test

A 50,000-transaction replay of the IBM AML dataset is run at 1,000 transactions per second to simulate peak bank load. P50, P95, and P99 latency percentiles are recorded. Any stage exceeding its budget triggers targeted profiling using Python `cProfile` or `py-spy`.

5.3. FIU Evidence Package Validation

- SAR narratives are manually reviewed against the corresponding IBM AML ground-truth labels to confirm accuracy and regulatory citation correctness.
- PMLA section citations are validated against the ChromaDB retrieval results to ensure grounding (no hallucinated citations).
- Package generation time is measured from alert confirmation trigger to download availability. Target: < 5 seconds.

5.4. Compliance Validation

- Verify that no PII fields (name, Aadhaar, PAN) are stored in graph edge attributes or ChromaDB documents.
- Confirm that all Account Aggregator data pulls include a valid consent token in the request log.
- Validate that the SQLite audit DB records a complete reasoning chain for every Gemini copilot query (required for regulatory review under PMLA reporting obligations).

6. References

- [1] Lin, J. et al. (2024). *FraudGT: A Simple, Effective, and Efficient Graph Transformer for Financial Fraud Detection*. ACM ICAIF 2024, pp. 292–300.
- [2] Lopez-Rojas, E.A. & Axelsson, S. (2014). *BankSim: A Bank Payment Simulation for Fraud Detection Research*. ECMS 2014.
- [3] Kosowski, A. et al. (2025). *The Dragon Hatchling: The Missing Link Between the Transformer and Models of the Brain*. arXiv preprint.
- [4] IBM Research (2023). *IBM AML Anti-Money Laundering Dataset (AML-HI-Large): 2.1M nodes, approximately 180M transactions*.
- [5] Reserve Bank of India (2023). *Framework for Real-time Fraud Risk Monitoring (FRM)*.
- [6] Ministry of Electronics and Information Technology (2023). *Digital Personal Data Protection Act 2023*.
- [7] Reserve Bank of India / Sahamati (2016, amended 2021). *Account Aggregator Framework*.